

4. Collections, Iteration, and Object Structure

Author -- Yucheng Wu

Version -- 26.1.25

Example 4.1 — Chapter Overview Example

```
import java.util.ArrayList;
import java.util.Iterator;

public class Demo {
    public static void main(String[] args) {
        ArrayList<String> items = new ArrayList<>();

        items.add("apple");
        items.add("banana");
        items.add("orange");

        // for-each loop (definite iteration)
        for (String item : items) {
            System.out.println(item);
        }

        // while loop with index (indefinite iteration)
        int index = 0;
        while (index < items.size()) {
            System.out.println(items.get(index));
            index++;
        }

        // iterator usage
        Iterator<String> it = items.iterator();
        while (it.hasNext()) {
            String s = it.next();
            if (s.startsWith("b")) {
                it.remove();
            }
        }
    }
}
```

Example 4.2

```
import java.util.ArrayList;
/**
 * A class to hold details of audio files.
 *
 * @author David J. Barnes and Michael Kölling
 * @version 7.0
 */
public class MusicOrganizer
{
    // An ArrayList for storing the file names of music files.
    private ArrayList<String> files;
```

```

/**
 * Create a MusicOrganizer
 */
public MusicOrganizer()
{
    files = new ArrayList<>();
}

/**
 * Add a file to the collection.
 * @param filename The file to be added.
 */
public void addFile(String filename)
{
    files.add(filename);
}

/**
 * Return the number of files in the collection.
 * @return The number of files in the collection.
 */
public int getNumberOfFiles()
{
    return files.size();
}

/**
 * List a file from the collection.
 * @param index The index of the file to be listed.
 */
public void listFile(int index)
{
    if(index >= 0 && index < files.size()) {
        String filename = files.get(index);
        System.out.println(filename);
    }
}

/**
 * Remove a file from the collection.
 * @param index The index of the file to be removed.
 */
public void removeFile(int index)
{
    if(index >= 0 && index < files.size()) {
        files.remove(index);
    }
}
}

```

4.1. Collection

Definition

A collection is an object that stores and manages a group of elements.

Features

- Stores multiple objects in one structure
- Size can change dynamically
- Elements are objects, not primitive values

Example

```
ArrayList<String> list = new ArrayList<>();  
list.add("A");  
list.add("B");
```

- `ArrayList` is a common collection type.

Remark

- Collections reduce manual array management
 - Part of the Java Collections Framework
-

4.2. Library and Import Statement

Definition

A library is a set of pre-written classes.

An import statement allows access to library classes.

Features

- Avoids rewriting common code
- Organized into packages

Example

```
import java.util.ArrayList;
```

- Makes `ArrayList` available.

Remark

- Must appear at the top of the file
 - `java.lang` is imported automatically
-

4.3. Package

Definition

A package groups related classes under a namespace.

Features

- Prevents name conflicts
- Improves code organization

Example

```
java.util.ArrayList
```

- `java.util` is the package name.

Remark

- Packages reflect directory structure
-

4.4. Index of Collection

Definition

An index is a zero-based integer position of an element in a collection.

Features

- First element index is `0`
- Last index is `size() - 1`

Example

```
String s = list.get(0);
```

- Accesses the first element.

Remark

- Invalid index causes `IndexOutOfBoundsException`
-

4.5. Definite Iteration

Definition

Definite iteration repeats a known number of times.

Features

- Iteration count is fixed
- Used when all elements must be processed

Remark

- Common with collections
-

4.5.1. For-Each Loop

Definition

A loop that processes each element in a collection once.

Features

- No index variable
- Automatically stops

Example

```
for (String s : list) {  
    System.out.println(s);  
}
```

- `s` represents each element.

Remark

- Cannot change collection structure
 - Preferred for readability
-

4.6. Indefinite Iteration

Definition

Indefinite iteration repeats until a condition becomes false.

Features

- Number of repetitions is unknown
- Condition-controlled

Remark

- Risk of infinite loops
-

4.6.1. While Loop

Definition

A loop that executes while a boolean condition is true.

Features

- Condition checked before each iteration
- May execute zero times

Example

```
int i = 0;
while (i < list.size()) {
    System.out.println(list.get(i));
    i++;
}
```

Remark

- Requires correct condition and update
-

4.6.2. While Loop with Index

Definition

Uses an index variable to access collection elements.

Features

- Manual control of position
- Supports partial traversal

Example

```
int index = 0;
while (index < list.size()) {
    System.out.println(list.get(index));
    index++;
}
```

Remark

- More flexible than for-each
-

4.7. Iterator

Definition

An iterator is an object used to traverse a collection safely.

Features

- Works with all collection types

- Maintains internal position

Example

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

Remark

- Independent of index access
-

4.7.1. Iterator Methods

Definition

Standard methods for traversal.

Features

- `hasNext()` — checks remaining elements
- `next()` — returns next element

Remark

- Calling `next()` without `hasNext()` risks exception
-

4.7.2. Removing Elements with Iterator

Definition

Safe removal during iteration using `Iterator.remove()`.

Features

- Prevents concurrent modification error
- Removal affects current element only

Example

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("A")) {
        it.remove();
    }
}
```

Remark

- Forbidden in for-each loops
-

4.8. Anonymous Object

Definition

An anonymous object is created without assigning it to a variable.

Features

- Used once
- No reference stored

Example

```
list.add(new String("hello"));
```

Remark

- Common in method calls
 - Reduces variable clutter
-

4.9. Index vs Iterator vs For-Each

Definition

Different ways to traverse collections.

Features

- **Index**: flexible, ArrayList-specific
- **For-each**: simple, definite iteration
- **Iterator**: general, supports removal

Remark

- Choose based on task requirements